13.Pre-processore

Il Pre-processore di c è un macro processore che è usato automaticamente dal compilatore di C per trasformare il tuo programma prima dell'attuale compilazione.

Il pre-processore C fornisce quattro strutture distinte che possono essere utilizzate a piacimento:

- inclusione del file header
- Macro espansione
- Compilazione condizionale

Include

un `#include` un istruzione del pre-processore dove inserisce i contenuti di uno specifico file nel parte del file desiderato, e ne esistono due tipi :

- `#include` <math.h> , cosi importiamo delle libreria standard
- `#include` "myproject.h" , importiamo dei file specifici nel nostro programma

Definire e usare i macros

In C puoi usare e creare i macros usando il pre-processore `directive` , un comune utilizzo è quello di definire il nome di una variabile per una costante numerale

```
#define ARRAY_SIZE 100;
double data[ARRAY_SIZE];
```

prima di compilare un file , il pre processore elimina tutte le righe commentate esegue le direttive ed espande le macro nei file sorgenti

Normalmente GCC non lascia alcun file di output intermedio contenente i risultati di questa fase di pre-laborazione.

Pre processing

è un macro processore che prepara il codice prima della compilazione vera e propia

- Include : inserisce il contenuto di un file header.
  - `#include` <file.> : cerca libreria di sistema
  - `#include` "file" : cerca prima nella cartella corrente
- Macro : sostituzione di testo per costanti o abb.
- **Flag** (-E) = ferma GCC dopo questa e mostra codice espanso

Compilatore

Assembly

Al cuore del lavoro del compilatore c'è la traduzione dei programmi in C in linguaggio assembly, questo linguaggio vicino a livello macchina ma comprensibile anche all'uomo

-S

Possiamo usare l'opzione -S per fermare il processo in compilazione dopo , se non specifichiamo il file output , GCC -S crea un file assembly con il nome che termina in .s per ogni file input compilato

Assembler

Traduce il file Assembly (.s) in un file Oggetto (.o) , dove conterrà il codice macchina non ancora eseguibile perché mancano i collegamenti esterni

- **flag** = *-c* istruisce gcc a fermarsi qui , producendo solo file .o

Linking

Ultimo passaggio , unisce tutti i file .o , e le librerie di sistema , per creare un unico file **ESEGUIBILE** e risolve i riferimenti esterni (collega le chiamate di funzione alla loro definizione)

GDB

GDB è un debugger a riga di comando simbolico. "Simbolico" significa che puoi fare riferimento a variabili e funzioni nel programma in esecuzione utilizzando i nomi che hai assegnato loro nel codice sorgente C.

Per visualizzare e interpretare questi nomi, il debugger necessita di informazioni sui tipi delle variabili e delle funzioni nel programma, oltre a sapere quali istruzioni nel file eseguibile corrispondono a quali righe nei file sorgente.

Flag = -g fondamentale, questa aggiunge simboli di debug , senza essa non viene inclusa la "Symbol Table" e GDB non può mostrarci i nomi delle variabili
Breakpoint (b) = serve a dire al debugger dove fermarsi
Next(n) vs Step(s) :

- n = esegue la riga successiva , se ce una funzione , la esegue tutta intera e passa oltre
- s = esegue riga successiva , se ce una funzione , entra dentro essa per debuggarla riga per riga